Final Report

Product Design Review（PDR）Core Package

專題題目：以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化(Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link)

---

1. PDR Executive Summary

本專題的目標在於將「單一瓶頸鏈路（Single Bottleneck Link）的壅塞控制」形式化為一個深度強化學習（Deep Reinforcement Learning, DRL）的序列決策問題。研究問題聚焦於探討：在高度變動的網路環境中，如何讓 DRL Agent 學會根據當下網路狀態（如吞吐量、延遲、丟包率），動態調整傳輸策略，以達到吞吐量最佳化與延遲、丟包之間的合理權衡（Trade-off），而非盲目最大化單一指標。

為什麼需要 PDR（Product Design Review 產品設計規格書）？
在學術專題或系統工程中，從「概念提案（Proposal）」直接跳入「程式實作（Implementation）」往往會帶來極高的偏航風險與範圍失控（Scope creep）。本 PDR 的存在是作為概念與實作之間的橋樑，將抽象的研究想法具象化為可驗證、可執行的規格基準（Source of truth）。

PPT 如何轉成 Spec-driven Development Pipeline？
本專題嚴格遵循「規格驅動開發（Spec-driven Development）」的 Pipeline：`PPT → PDF → NotebookLM → PDR 報告/資訊圖表 → OpenSpec → task.md → code → Antigravity AI`。在此流程中，初版的 PPT 提案被萃取並轉化為本份 PDR，接著此 PDR 將作為後續 OpenSpec 拆解任務（如 01-project-charter、02-ns3-baseline-benchmark 等）的主要 source-of-truth。Antigravity 將嚴格依照規格執行，形成 Plan → Apply → Verify 的閉環。

文件定位：本文件為「Proposal-stage PDR / OpenSpec-ready 規格草案」，不包含任何 Detail Implementation、不進行神經網路訓練、且絕對不假裝已有任何實驗結果。所有的成效均為「預期發現（Expected findings）」與「規劃中的評估（Planned evaluation）」。

---

2. PDR 主報告 (Product Design Review)

(以下為可匯出為 PDF 供後續 OpenSpec 使用之 PDR 報告標準目錄與結構設計)

本節為可匯出為 PDF 的正式 PDR 主報告本文。內容依照期末專題提案與產品設計規格書（Specification）的標準撰寫，全面採用規劃（Planned）、預期（Expected）的語氣，確保無假造實驗結果、無實作細節與程式碼，並將範圍嚴格鎖定在單一瓶頸鏈路。

1. Project Overview

本專案（Project）旨在將網路傳輸中的「壅塞控制（Congestion Control）」形式化為一個深度強化學習（Deep Reinforcement Learning, DRL）的序列決策問題。研究的具體場景鎖定於「單一瓶頸鏈路（Single Bottleneck Link）」，計畫由傳送端（Sender-side）的 DRL Agent 根據觀測到的網路狀態，動態調整傳輸控制策略。

本專案的定位為期末專題提案所衍生的規格驅動開發上游文件。專案目標不在於盲目最大化單一的吞吐量數據，而是建立一個可重現（Reproducible）的模擬環境，探討 DRL 在網路吞吐量（Throughput）、往返延遲（RTT）與封包遺失率（Packet Loss）之間的動態權衡（Trade-off）能力。

2. Problem Background and Objective Evidence

在全球網路數據爆炸性成長的背景下（如 Cloudflare 報告指出 2024 年全球網路流量成長 17.2%，Ericsson 預測 5G 行動數據佔比將於 2031 年達 83%），底層網路吞吐量的最佳化與壅塞控制仍是極具價值的基礎工程挑戰。然而，傳統 TCP 壅塞控制主要基於固定規則或啟發式邏輯（如 RFC 5681 定義的慢啟動、壅塞避免等階段），在面對高度變動的網路條件時，其適應性具有侷限。

根據 Mathis 模型（$\text{Throughput} \lesssim \frac{MSS}{RTT \sqrt{p}}$），TCP 吞吐量受到 RTT 與丟包率的嚴格數學約束。若傳輸協定僅以最大化吞吐量為單一目標，往往會導致瓶頸節點的緩衝區滿載，引發嚴重的佇列延遲（Bufferbloat）與後續的大量丟包。因此，現代壅塞控制實為一個高度複雜的權衡問題，需要更具動態適應力的決策框架介入。

3. Research Problem and Scope

本研究的核心問題為：「如何將單一瓶頸鏈路的壅塞控制形式化為深度強化學習問題，並與傳統 TCP Baseline 比較其在吞吐量、RTT 與丟包率之間的權衡能力？」 為了確保本提案的可執行性（Executability），專案範圍將受到最嚴格的邊界控管。

本專案計畫將實作範圍完全限制在「單一傳送端至接收端之瓶頸鏈路模擬」。為確保本提案可在課程時程內完成，本研究將範圍聚焦於單一瓶頸鏈路模擬；IPFS / P2P 覆蓋網路、QUIC 協定、Kernel-level TCP、多智慧體控制與真實網際網路部署，將保留為未來延伸方向，而非本 proposal 階段實作範圍。

4. A-to-F Proposal Logic

本提案遵循嚴謹的 A-to-F 學術邏輯：

- **Attention / Motivation**: 現代分散式系統高度依賴穩定傳輸，但基礎仍受限於瓶頸鏈路的吞吐能力。
- **But**: 傳統 TCP 面臨吞吐量、延遲與丟包的 Trade-off，且盲目追求高吞吐易引發 Bufferbloat。
- **Cure**: 提出以 DRL 將壅塞控制重新建模為依據網路狀態動態適應的序列決策問題。
- **Development**: 規劃採用 ns-3 網路模擬器與 ns3-gym 介面，搭配 Stable-Baselines3 建立 DQN 原型。
- **Experiments**: 規劃於不同延遲、頻寬與干擾情境下，全面評估 DRL 與 CUBIC、NewReno 等 Baseline 之差異。
- **Findings**: 預期產出客觀的 Trade-off 分析結果，而非武斷宣稱 DRL 能無條件超越所有傳統演算法。

5. Related Work Groups

本專案的相關研究群被結構化為四個主要象限，以釐清本計畫的技術定位：

1. **Traditional TCP Congestion Control**: 包含 NewReno、CUBIC 與 BBR，本群組提供成熟的規則式/模型式演算法，作為本提案未來驗證的比較基準（Baselines）。
2. **Learning-Based Congestion Control**: 以 Aurora 等研究為代表，證明將壅塞控制視為連續決策問題是可行的正式研究路線，提供本專案的學術正當性。
3. **Network Simulation and RL Interface**: 包含 ns-3、ns3-gym 與 SB3，證明在 C++ 封包模擬與 Python 訓練環境之間建立橋樑的架構具備技術可行性。
4. **Benchmark and Reproducibility**: 借鑑 Pantheon 的精神，確立本專題應重視公平測試介面與可重現性（但 Pantheon 本身不列為第一版 MVP 的強制安裝依賴）。

6. Proposed Design Specification

本設計規格規劃建立一套可重現的模擬式工作流（Reproducible simulation-based workflow）。系統底層計畫採用 ns-3 作為網路模擬器（Simulator），負責處理真實的封包傳遞與佇列動態；中間層採用 ns3-gym 作為標準的 OpenAI Gym 強化學習介面，連接 C++ 與 Python 環境；上層決策端則採用 Stable-Baselines3 框架建構 DRL Agent。

為降低實作初期的不確定性，最小可行性產品（MVP）的 Agent 演算法規劃採用深度 Q 網路（DQN）。DQN 原生適合處理離散動作空間（Discrete Action Space），預期能最快建立具備基礎學習能力的雛形系統，而 PPO 等連續動作演算法則保留為未來的加分擴充項目。

7. MDP Formalization

本研究正式將壅塞控制建模為馬可夫決策過程（Markov Decision Process, MDP），並定義為五元組 $M = (S, A, P, R, \gamma)$。
在此框架下，Agent 作為傳送端的控制器，透過不斷觀測網路狀態來下達動作指令。狀態轉移機制（Transition Dynamics, P）將完全交由 ns-3 模擬器內部的實體網路動態（如物理傳播延遲、緩衝區排隊法則）來決定，Agent 無法破壞物理限制。
折扣因子（Discount Factor, γ）則規劃用於控制 Agent 對於長期傳輸穩定性與短期吞吐量爆發的重視程度。

8. State / Action / Reward Specification

針對 MDP 的實體規格，本專題提出以下初始設定：

- **State Space (S)**: 規劃採用最小可行且具解釋性的狀態特徵，包含：近期吞吐量（Recent Throughput）、往返延遲（RTT）、封包遺失訊號（Loss Signal）、壅塞指標（Congestion Indicator/Queue）以及上一步的動作（Previous Action）。
- **Action Space (A)**: 規劃採用受限的離散控制動作（Discrete Control），具體定義為：減少（Decrease）、維持（Keep）、增加（Increase）傳送速率或壅塞視窗。
- **Reward Function (R)**: 獎勵函數設計為權衡公式 $r_t = \alpha T_t - \beta RTT_t - \lambda L_t$。此設計旨在給予高吞吐量（$T_t$）正向獎勵的同時，強制針對高延遲（$RTT_t$）與丟包（$L_t$）施加懲罰，以避免 Agent 學出導致 Bufferbloat 的極端策略。

9. Baselines and Metrics

為了客觀評估 DRL Agent 的行為，本專案規劃導入三個業界標準的 TCP 演算法作為基準線（Baselines）：代表 Loss-based 邏輯的 NewReno、代表 Heuristic 邏輯且廣泛使用的 CUBIC，以及代表 Model-based 頻寬延遲估計的 BBR。

在成效量測指標（Metrics）部分，本專案將放棄「唯吞吐量論」。計畫評估的量化維度包含：平均吞吐量（Average Throughput）、平均往返延遲（Average RTT）、封包遺失率（Packet Loss Rate），以及結合三者的綜合效用分數（Utility Score）。此外，亦將觀測 DRL 專屬的訓練獎勵曲線（Reward Curve）與收斂行為（Convergence Behavior）。

10. Planned Experiment Matrix

本專題規劃了一套實驗矩陣（Experiment Matrix），預期在四種不同挑戰等級的瓶頸情境下檢驗 Agent 的適應能力：

1. **S1 穩定低延遲（Stable low-delay bottleneck）**：基礎測試，評估 Agent 是否能學會基本傳輸並與 Baseline 表現持平。
2. **S2 穩定高延遲（Stable high-delay bottleneck）**：長管測試，預期觀察 Reward 設計是否能有效抑制深佇列（Deep queueing）的產生。
3. **S3 變動頻寬（Variable bandwidth bottleneck）**：適應力測試，旨在分析 DRL Agent 面對可用容量動態縮放時的反應速度與容錯能力。
4. **S4 交叉流量干擾（Bottleneck with cross traffic）**：高階測試，規劃加入外部未知的背景流量，評估控制策略的魯棒性（Robustness）。

11. Risk and Fallback Strategy

為確保本規格書具備極高的工程可行性，本專案已建立主動的風險降階計畫（Fallback Plan）。若在建置期遭遇 ns3-gym 與系統環境的版本匹配問題，將退回使用純 Python 的簡化 Gym Bottleneck 環境；若 DQN 模型初期無法順利收斂，將進一步簡化 State Space 甚至限縮 Action 的影響幅度。

同時，為了防範 AI 輔助開發工具（如 Antigravity）在實作過程中發生範圍偏航（Scope drift），本專案強制規定所有實作必須嚴格遵守 OpenSpec 任務拆解。若後續出現 IPFS 實作、多路徑路由或其他超出 MVP 的延伸需求，將先回到 PDR 與 OpenSpec 進行 scope review，再決定是否列入 future work。

12. PDR-to-OpenSpec Roadmap

本計畫嚴格拒絕「從概念直接跳入撰寫程式碼」的開發模式。本 PDR 報告將作為後續開發的「主要規格真相來源（source-of-truth）」，並計畫拆解為五個循序漸進的 OpenSpec 變更任務（Changes）。
具體推演路徑為：首階透過 `01-project-charter` 凍結範圍與目標；次階透過 `02-ns3-baseline-benchmark` 建立傳統 TCP baseline 的模擬與資料輸出流程，並驗證是否能產生可分析的 log / figures；隨後進行 `03-ns3-gym-environment` 的環境打通與 `04-dqn-mvp-agent` 的模型訓練；最終由 `05-reporting-figures-demo` 負責產出可供展示的綜合效能報告。全流程將遵循 Plan $\rightarrow$ Apply $\rightarrow$ Verify 的嚴謹閉環。

13. Expected Contributions and Next Steps

本專案的預期發現（Expected Findings）並非武斷證明 DRL 能在所有網路環境中擊敗發展數十年的 CUBIC 或 BBR，而是期望確立一套清晰、可重現的 DRL 壅塞控制問題框架。本專案的預期貢獻在於產出涵蓋 throughput、RTT 與 loss 多維度指標的客觀 Trade-off 分析報告。

作為本專案的 Next Steps，計畫將本 PDR 規格全面對接至 Antigravity 平台，依序啟動 `change-01` 至 `change-02` 的基線環境部署。本研究亦期望透過完善此單一瓶頸鏈路的底層傳輸優化前置作業，為未來針對大型分散式系統（如 IPFS 或 P2P 網路）的傳輸協定研究，奠定堅實的領域知識與模擬技術基礎。

---

3. Objective Evidence Table

Evidence category	Source group	Key evidence	How it supports the proposal	Suggested slide or PDR section
Traffic growth	Cloudflare / Ericsson Reports	Global Internet traffic grew 17.2% in 2024; 5G mobile data share expected to reach 83% by 2031.	證明資料傳輸規模龐大，底層吞吐量最佳化與壅塞控制仍為具高度價值之基礎問題。	Introduction / Slide 2
Rule-based TCP congestion control	RFC 5681 / ns-3 TCP docs	傳統 TCP 控制仰賴 Slow start, congestion avoidance, fast retransmit, fast recovery 等固定狀態機與規則。	定位既有技術很成熟但屬「規則式」，帶出將其轉化為「動態決策學習」的空間。	Problem / Slide 5
TCP throughput affected by RTT and loss	Mathis Model	$\text{TCP Throughput} \lesssim \frac{MSS}{RTT \sqrt{p}}$ (Throughput 受制於延遲與丟包率平方根)。	數學層面上證實 Throughput 不是獨立變數，必須與 RTT 和 Loss 進行整體權衡考量。	Core Observation / Slide 3

Evidence category	Source group	Key evidence	How it supports the proposal	Suggested slide or PDR section
Bufferbloat / queueing delay	BBR / RFC 7567 (ACM Queue)	盲目提升吞吐量會導致瓶頸節點的 Buffer 滿載，引發極高的佇列延遲 (Bufferbloat)。	強調單一最大化吞吐量是錯誤目標，合理化 Reward Function 必須設計延遲懲罰。	Trade-off Problem / Slide 4
DRL as sequential decision formulation	Aurora (PMLR) / RL Literature	壅塞控制可視為 Learning-based control problem，由 Agent 觀測網路狀態後下達速度調整指令。	確立 DRL 應用於本題的學術正當性，證實 Cure (解法) 在基礎邏輯上是成立的。	Gap & Objective / Slide 6

4. Grouped Related Work Table

Group	Role in this project	Key sources	What this group contributes	What this group does NOT imply
1. Traditional TCP Congestion Control	Baseline 比較基準	NewReno, CUBIC, BBR, RFC 5681	確立成熟的規則式/啟發式通訊協定，作為後續評估 DRL 是否能學到有效 Trade-off 的比較基準。	本專題「不」進行核心層級 (Kernel-level) TCP 修改，僅在模擬器中取用。
2. Learning-Based Congestion Control	研究框架合理性	Aurora, DRL CC related literature	證明將壅塞控制視為連續決策問題是可行的正式研究路線，支持本研究 DRL 建模的有效性。	本專題「不」直接完全複刻該篇論文的所有複雜架構，而是收斂於單一瓶頸 MVP。
3. Network Simulation and RL Interface	開發工具鏈與架構	ns-3, ns3-gym, Stable-Baselines3	確保系統架構可行性，ns-3 提供穩定的網路動態模擬，ns3-gym 提供標準 OpenAI Gym 橋接。	這「不」代表我們會直接將訓練結果部署至真實網路 (Real Internet deployment)。
4. Benchmark and Reproducibility	實驗精神與評估哲學	Pantheon, benchmark reproducible philosophy	指導本專題應採用統一的測試介面、公平的矩陣情境以及重視可重現性 (Reproducibility)。	Pantheon「不是」本專題第一版 MVP 的強制安裝依賴 (Mandatory dependency)。

5. PDR-ready Proposed Design Table

Component	Specification	Rationale	Input	Output	Acceptance criteria	OpenSpec change mapping
Scenario	Single sender → bottleneck link → receiver	降低實作變數，專注驗證單一壅塞點的控制邏輯。	Topology params (Bandwidth, Delay, Queue)	Simulated network traffic flows	成功於 ns-3 建構包含指定容量瓶頸之單一流拓樸。	`02-ns3-baseline-benchmark`
Simulator	ns-3	業界與學術標準，原生支援多種 TCP baseline 模型。	Scenario configs	Log traces (.pcap, flow stats)	穩定運行傳輸且無 Segfault 崩潰。	`02-ns3-baseline-benchmark`
RL interface	ns3-gym	提供 C++ 網路模擬與 Python 訓練環境間的標準通訊橋樑。	RL Action	State / Observation / Reward	ns-3 成功實例化 Gym Environment 並能穩定 step/reset。	`03-ns3-gym-environment`
Agent	DRL congestion-control agent (MVP: DQN)	DQN 適合處理離散動作空間，易於建立第一版可訓練 MVP。	Normalized State vector	Discrete Action (int)	後續將檢查 Agent 是否出現可觀察的 reward curve 變化，並評估 Policy 是否能穩定輸出 Action 決策。	`04-dqn-mvp-agent`
State	Recent throughput, RTT, loss, congestion indicator, prev action	最小化且具解釋性的狀態特徵，避免複雜維度影響初期收斂。	Raw ns-3 statistics	Normalized Float Array	每個 step 皆能正確回傳更新後的 State 特徵。	`03-ns3-gym-environment`
Action	Decrease, Keep, Increase sending rate / cwnd	離散化動作最安全，降低 Action Scaling 導致的訓練崩潰風險。	Policy network prediction	Control signal to ns-3 sender	動作能有效影響 ns-3 內部的發送速率或視窗大小。	`03-ns3-gym-environment`
Reward	Trade-off: throughput	引導 Agent 學會避開 Bufferbloat，	State metrics	Scalar float	依據公式正確計算並回傳即時獎勵數值。	`03-ns3-gym-environment`

Component	Specification	Rationale	Input	Output	Acceptance criteria	OpenSpec change mapping
	positive, RTT/loss penalty	達成健康網路權衡。				
Baselines	NewReno, CUBIC, BBR	ns-3 內建且代表了 Loss-based, Heuristic, Model-based 三種經典範式。	Same scenario configs	Baseline perf logs	成功繪製三者的吞吐量、延遲對比基準線圖。	`02-ns3-baseline-benchmark`
Metrics	Throughput, RTT, loss rate, utility, reward curve	多維度驗證，不單看吞吐量。	Evaluation logs	Tables / Charts	產生可供對比與視覺化的綜合評估數據。	`05-reporting-figures-demo`
MVP Boundaries	baseline + ns3-gym env + DQN prototype	最低限度驗證規格，證明架構可運行且方法有效。	System start command	Trained agent + basic plots	涵蓋環境搭建至單一模型收斂的全流程。	`04-dqn-mvp-agent`
Scope Exclusions	IPFS, QUIC, kernel TCP, multi-agent, real deployment	專案範圍保護，防範 AI 或開發過程偏航 (Scope drift)。	N/A	N/A	無任何牽涉排除項目之多餘程式碼引入。	`01-project-charter`
Reporting	PPT, PDF report, README, NotebookLM Infographic	符合期末專題交付規定，完成溝通閉環。	Data logs, Training curves	Formatted academic materials	能順利向教授進行 10 分鐘的成果規劃簡報。	`05-reporting-figures-demo`

6. MDP Specification

1. 正式規格版

本專題將單一瓶頸鏈路的壅塞控制形式化為馬可夫決策過程 (Markov Decision Process, MDP)，定義為五元組 $M = (S, A, P, R, \gamma)$：

- **S (State Space)**: $s_t = [T_t, RTT_t, L_t, C_t, a_{t-1}]$
  包含觀測之近期吞吐量 (Throughput)、往返延遲 (RTT)、丟包訊號 (Loss rate)、壅塞指標 (Congestion indicator/queue occupancy) 及上一步動作 (Previous action)。
- **A (Action Space)**: $a_t \in \{Decrease, Keep, Increase\}$
  離散傳輸控制動作，分別對應減少、維持、增加傳送速率或壅塞視窗 (cwnd)。
- **P (Transition Dynamics)**: $P(s_{t+1} | s_t, a_t)$
  由 ns-3 網路模擬器內部的實體網路動態（封包排隊、傳播延遲等物理定律）所決定的狀態轉移機率。
- **R (Reward Function)**: $r_t = \alpha T_t - \beta RTT_t - \lambda L_t$
- **γ (Discount Factor)**: 數值範圍 $\gamma \in [0, 1)$，用於計算未來累積獎勵的折扣。

Reward 公式變數解釋：

- **r_t**: 在時間點 t Agent 獲得的即時獎勵分數。
- **T_t**: 觀測到的吞吐量 (Throughput)。
- **RTT_t**: 觀測到的往返延遲 (Round-Trip Time)。
- **L_t**: 觀測到的丟包率或丟包數量 (Packet Loss)。
- **α**: 鼓勵吞吐量成長的獎勵權重係數。
- **β**: 懲罰高延遲的權重係數。
- **λ**: 懲罰網路丟包的權重係數。
- **γ**: 專屬於 MDP 用以決定重視長期或短期利益的折扣因子（嚴格聲明：γ 僅作為 Discount factor，不與丟包權重 λ 混用）。

2. 白話說明版

- **State (我看見什麼)**: Agent 在每個瞬間「看」到目前的傳輸速度有多快、延遲有多高、有沒有發生封包掉落，以及剛才自己下了什麼指令。
- **Action (我能做什麼)**: Agent 只能決定這一步要「踩煞車 (變慢)」、「穩住油門 (維持)」，還是「踩油門 (加速)」。
- **Transition (物理規則)**: Agent 下達指令後，封包會真實進入網路水管中擠壓。水管多粗、有多長，是由 ns-3 這個「虛擬物理世界」決定，Agent 無法破壞規則。
- **Reward (我拿幾分)**: 設計核心是「傳得越快分數越高，但如果害大家塞車（延遲變高）或把東西弄丟（丟包），就會被嚴厲扣分」。這強迫 Agent 必須成為一個遵守交通規則的好司機，而不是飆車族。
- **Discount Factor (我的眼光)**: γ 決定了 Agent 是一個短視近利（只看眼前這一步）還是深謀遠慮（在乎整趟傳輸順暢）的決策者。

7. Planned Experiment Matrix

本計畫將評估 DRL 在多元網路挑戰下的適應性，以下為規劃中的四組核心實驗情境 (Scenarios)。*(註：所有觀察結果均為「預期將評估」之規劃，不代表已完成實作。)*

Scenario ID	Bandwidth	Propagation delay	Queue setting	Traffic pattern	Baselines	Metrics	Expecte observat
S1: Stable low-delay bottleneck	Fixed (e.g., 10M)	Low (e.g., 10ms)	Medium	Single flow	NewReno / CUBIC / BBR	Throughput, RTT, Loss, Utility	預期將評 Agent 是 利學習基 規則，且 期能貼近 Baseline
S2: Stable high-delay bottleneck	Fixed (e.g., 10M)	High (e.g., 100ms)	Medium	Single flow	NewReno / CUBIC / BBR	Throughput, RTT, Loss, Utility	預期觀察 遲下，Re 的 RTT 是否能有 Agent 產 列行為 (D queueing
S3: Variable bandwidth bottleneck	Variable (e.g., 5-20M)	Medium	Medium	Single flow	NewReno / CUBIC / BBR	Throughput, RTT, Loss, Utility	預期分析 於環境動 的適應力 其是否能 整發送率 成大量遺
S4: Bottleneck with cross traffic	Fixed / Variable	Medium	Small / Medium	Main flow + cross traffic	NewReno / CUBIC / BBR	Throughput, RTT, Loss, Utility	預期評估 受到未知 背景流量 時，是否 足夠的穩 魯棒性 (Robustr

8. Risk Register and Fallback Plan

為了確保本提案具備極高的可執行性 (Executability)，在此正式建立系統開發風險表。本表旨在向評審證明，所有潛在的開發中斷均已有明確的降階 (Fallback) 或迴避策略。

Risk ID	Risk description	Probability	Impact	Why it matters	Prevention	Fallback	Affected OpenSp change
R01	ns3-gym version issue (版本相依問題)	高	高	C++ 與 Python 間的版本匹配極易發生建置失敗。	嚴格鎖定已知穩定的 Ubuntu OS 與 ns-3 LTS 版本進行環境建置。	退回使用純 Python 的簡化 Gym Bottleneck Toy Environment。	`03-ns3-environm`
R02	BBR dependency in	中	中	BBR 在特定較舊的 ns-3 版本中	優先清查 ns-3 官方 TCP model	MVP 的 Baseline 比較僅	`02-ns3-baseline benchmar`

Risk ID	Risk description	Probability	Impact	Why it matters	Prevention	Fallback	Affected OpenSp change
	ns-3 (BBR 模組依賴問題)			可能未內建或需額外 Patch。	支援列表。	保留 CUBIC 與 NewReno，BBR 降級為加分項。	
R03	DQN non-convergence (模型不收斂)	中	高	強化學習在連續時間序與複雜獎勵下可能無法學到有意義的 Policy。	從最簡單的 State 特徵與極度受限的離散 Action Space 開始。	將實驗重點轉向「Trade-off 失敗原因分析」或與 Random agent 比較，不盲目追求超越 TCP。	`04-dqn-agent`
R04	Reward causing high RTT / loss (學出飆車策略)	高	高	僅關注吞吐量的錯置獎勵會導致嚴重的 Bufferbloat 效應。	在 MDP 規範階段即嚴格引入 $\beta \cdot RTT_t$ 與 $\lambda \cdot L_t$ 作為懲罰項。	規劃 Reward Ablation 實驗，比較「唯吞吐量」與「綜合權衡」的行為差異。	`03-ns3-environm`
R05	Baseline logging incomplete (日誌擷取不全)	中	高	無法正確捕捉 RTT 或 Loss 會導致後續完全無法與 Agent 比較。	在實作 Agent 之前，必須先以 Change-02 強制完成 Log spec 的對齊與跑通。	初期可暫時退回僅擷取 Throughput 與 RTT 兩項核心指標，Loss 列為後補。	`02-ns3-baseline benchmar`
R06	Toolchain / spec drift (AI 實作偏航)	高	高	Antigravity AI 在自動實作時容易自由發揮，改變了原始研究範圍。	嚴格執行 Spec-driven 開發，每次只允許載入一個 OpenSpec Change 並驗收。	隔離工作區塊，拒絕偏航的 PR，手動修正 Acceptance Criteria 後重試。	`All Cha`
R07	Topic misunderstood as implementation-heavy (被誤解為 IPFS 實作)	中	中	聽眾若誤解，會對 P2P 系統細節提出無法在單一瓶頸實驗中回答的問題。	在簡報與文件中以 scope boundary 說明本 proposal 階段聚焦單一瓶頸鏈路，IPFS 僅作未來延伸脈絡。	以一句話說明本計畫是為未來 IPFS / P2P 類網路底層傳輸優化建立先期分析環境。	`01-proj charter`
R08	DRL not outperforming TCP baselines (效能未達預期)	高	中	預期 DRL Agent 無法在所有情境全面擊敗發展幾十年的 CUBIC/BBR。	事先確立「Evaluation Philosophy」為：重點在分析 Trade-off，而非打擂台。	如實記錄並分析 Agent 在何種邊界條件下開始劣於傳統啟發式演算法。	`05-repo figures-`

OpenSpec-ready and Antigravity Package

9. OpenSpec-ready Executive Summary

本節的目標是將前述 PDR 主報告與核心規格表，無縫轉化為可供 AI 協作開發工具（如 OpenSpec 與 Antigravity AI）使用的「規格包與開工指令」。這對應教授白板上的後續流程：`PDR → OpenSpec → task.md → code`。

PDR 如何轉成 OpenSpec-ready changes：
PDR 是一份宏觀的高階設計文件。為了讓 AI 能夠穩定執行，我們將 PDR 拆解為五個循序漸進的 OpenSpec Changes（01-project-charter 到 05-reporting-figures-demo）。每個 Change 都有明確的輸入、輸出、驗收標準（Acceptance criteria）與驗證方法，確保 AI 不會一次載入過多資訊而產生幻覺。

為什麼目前仍不是 implementation：
本節的核心精神是「規格驅動開發（Spec-driven Development）」。在尚未建立嚴格的防偏航規格（Boundary & Non-goals）與基準測試（Baseline benchmark）之前，直接跳入撰寫 DQN 程式碼會帶來極高的範圍失控風險。因此，這份 OpenSpec-ready 包目前僅是 Markdown 格式的「執行草案」，用以向教授證明我們具備系統化掌控 AI 開發流程的能力。

Antigravity 後續工作與閉環保留：
後續 Antigravity 將被嚴格限制為「Implementation Agent」，必須依照本文件產出的 `task.md` 草案順序逐一施工。在每一個任務節點，都必須完整執行教授白板所指示的 **Plan** → **Apply** → **Verify** 閉環：先提出實作計畫（Plan）、撰寫程式碼（Apply）、跑通測試驗收（Verify）後，才能推進到下一個 Change，從根本上杜絕偏題。

---

10. OpenSpec-ready Package

以下為根據本 PDR 轉換而成的 OpenSpec-ready 規格包草案（目錄結構與 Markdown 內容綱要），供後續匯入 `openspec-preview/` 使用。

```
openspec/
  project.md
  changes/
    01-project-charter/
      proposal.md
      design.md
      tasks.md
      specs/project-charter/spec.md
    02-ns3-baseline-benchmark/
      proposal.md
      design.md
      tasks.md
      specs/baseline-benchmark/spec.md
    03-ns3-gym-environment/
      proposal.md
      design.md
      tasks.md
      specs/rl-environment/spec.md
    04-dqn-mvp-agent/
      proposal.md
      design.md
      tasks.md
      specs/dqn-agent/spec.md
    05-reporting-figures-demo/
      proposal.md
      design.md
      tasks.md
      specs/reporting-demo/spec.md
```

Change 01: project-charter

- **Change purpose**: 凍結專題題目、核心目標與開發邊界，建立專案規格的「主要 source-of-truth」。
- **Scope**: 定義單一瓶頸鏈路拓樸、RL 決策框架、基礎 Baseline 清單與 PDR-ready 架構。
- **Non-scope**: 本 proposal 階段不納入 IPFS 實作、QUIC、kernel-level TCP、多智能體（multi-agent）、多路徑路由與真實環境部署；上述方向保留為 future work 或外部延伸。
- **Inputs**: `PDR Core Package`。
- **Outputs**: `README.md`（專案總覽）、`docs/project_charter.md`。
- **Acceptance criteria**: 所有開發範圍限制與預期貢獻被清晰條列，且無任何超出 Scope 之描述。
- **Tasks checklist**:
  - ☐ Plan: 確認 PDR 中的 Non-goals 列表。
  - ☐ Apply: 產出 `README.md` 初始化專案介紹。
  - ☐ Verify: 確認文件是否明確要求先完成 charter 與 baseline，再進入 DQN 相關工作。
- **Verification method**: 人工 Spec Owner 審閱 `README.md`，確認無偏題文字。
- **Fallback plan**: 若範圍描述模糊，退回 PDR 重新收斂。

Change 02: ns3-baseline-benchmark

- **Change purpose**: 在不引入任何強化學習元素的前提下，先跑通 ns-3 模擬器並建立傳統 TCP 基準線。
- **Scope**: 實作 ns-3 point-to-point 單一瓶頸拓樸；跑通 NewReno、CUBIC 與 BBR；記錄吞吐量、RTT 與丟包率。
- **Non-scope**: 不啟動 Python 環境，不載入 ns3-gym，不進行模型訓練。
- **Inputs**: `Change 01`，ns-3 C++ API 文件。

- **Outputs**: C++ 模擬腳本、Log 收集機制、基準線預期觀察圖表（佔位符或腳本）。
- **Acceptance criteria**: 能夠穩定執行 ns-3 腳本並輸出三個 Baseline 的 Throughput, RTT, Loss 的 log 檔案，無 Segfault。
- **Tasks checklist**:
  - ☐ Plan: 定義 ns-3 網路拓樸參數（S1~S4 情境）。
  - ☐ Apply: 撰寫 ns-3 baseline 腳本。
  - ☐ Verify: 執行腳本並確認 log 格式齊全。
- **Verification method**: 執行 `ns3 run baseline_script`，檢查終端機與 pcap/log 輸出。
- **Fallback plan**: 若 BBR 版本不相容，降級僅評估 CUBIC 與 NewReno。

Change 03: ns3-gym-environment

- **Change purpose**: 將 ns-3 網路模擬環境封裝為符合 OpenAI Gym 標準的強化學習介面。
- **Scope**: 實作 ns3-gym bridge；定義 State (throughput, RTT, loss, queue, prev_action)；定義 Discrete Action (decrease, keep, increase)；實作 Trade-off Reward ($r_t = \alpha T_t - \beta RTT_t - \lambda L_t$)。
- **Non-scope**: 不進行神經網路建構與收斂訓練。
- **Inputs**: `Change 02` C++ 腳本、MDP Specification。
- **Outputs**: `ns3gym_env.py`、C++ 端 `MyGymEnv` 介面實作。
- **Acceptance criteria**: Python 端能順利呼叫 `env.reset()` 與 `env.step()`，並正確回傳規範的 (state, reward, done, info) 格式。
- **Tasks checklist**:
  - ☐ Plan: 對齊 C++ 與 Python 間的 State / Action / Reward 傳遞格式。
  - ☐ Apply: 實作 ns3-gym 環境封裝與 Reward Normalization。
  - ☐ Verify: 使用 Random Agent 進行 Smoke test，確保不崩潰。
- **Verification method**: 運行 Random Agent 迴圈 1000 steps，確認記憶體無洩漏且 step 順暢。
- **Fallback plan**: 若 ns3-gym 編譯失敗，退回純 Python 的簡化版模擬環境（Bottleneck Toy Env）。

Change 04: dqn-mvp-agent

- **Change purpose**: 建立第一個具備學習能力的 DRL Congestion Control Prototype。
- **Scope**: 匯入 Stable-Baselines3；實作 DQN Agent；在穩定情境下訓練；輸出訓練 Reward 曲線。
- **Non-scope**: 不處理連續動作空間（排除 PPO），不追求在所有動態情境擊敗 TCP。
- **Inputs**: `Change 03` 可運作之 Environment。
- **Outputs**: `train_dqn.py`、`evaluate_agent.py`、預訓練模型檔（`.zip`）。
- **Acceptance criteria**: 能完成 DQN training / evaluation pipeline，並輸出 reward curve 與 action log 供後續分析；是否收斂列為評估結果，不在 proposal 階段預設。
- **Tasks checklist**:
  - ☐ Plan: 設定 SB3 DQN 超參數（learning rate, buffer size）。
  - ☐ Apply: 撰寫訓練迴圈與評估迴圈程式碼。
  - ☐ Verify: 訓練並檢視 Tensorboard 的 Reward 收斂狀態。
- **Verification method**: 檢查訓練日誌、reward curve 與 action log，評估是否出現穩定趨勢。
- **Fallback plan**: 若 Agent 不收斂，進一步簡化 State Space 特徵。

Change 05: reporting-figures-demo

- **Change purpose**: 將實作數據轉化為可供期末展示的圖表與報告。
- **Scope**: 處理 Change 02 與 Change 04 的 log；繪製 Throughput-Delay 權衡圖；產出綜合 Utility 比較表。
- **Non-scope**: 不修改或微調任何核心環境或網路參數。

- **Inputs**: Baseline logs、DQN evaluation logs。
- **Outputs**: `plot_results.py`、匯出之視覺化圖表 (`.png` / `.pdf`)。
- **Acceptance criteria**: 產出至少涵蓋「吞吐量對比」、「RTT 對比」與「Reward 曲線」之視覺化圖表。
- **Tasks checklist**:
  - ☐ Plan: 確定 4 個情境的繪圖需求。
  - ☐ Apply: 使用 Matplotlib/Seaborn 自動解析 log 並繪圖。
  - ☐ Verify: 確認圖例、坐標軸、單位標示正確無誤。
- **Verification method**: 人工檢查輸出圖表是否符合學術可讀性標準。
- **Fallback plan**: 若自動繪圖腳本失敗，退回人工使用 Excel 或簡單 Python script 繪製。

---

11. task.md 草案總表

以下總表確保 Antigravity 的開發節奏受到嚴格控管，必須依循相依性（Dependency）執行，不可躍進。

Change ID	Task ID	Task description	Required input	Expected output	Verification	Dependency	Prio
`01-charter`	`T-01-1`	建立專案 README 與架構文件	本 PDR	`README.md`	人工審閱 Non-goals	無	P1 (
`02-baseline`	`T-02-1`	實作 ns-3 單一瓶頸拓樸	PDR Proposed Design	ns-3 C++ 腳本	編譯並運行無報錯	`01-charter`	P1
`02-baseline`	`T-02-2`	擷取 NewReno, CUBIC, BBR 日誌	ns-3 TCP models	`.pcap` 或 flow logs	檢查 log 是否含 Tput, RTT, Loss	`T-02-1`	P1
`03-env`	`T-03-1`	實作 ns3-gym C++ to Python Bridge	PDR MDP Spec	`ns3gym_env.py`	成功啟動 Gym Environment	`02-baseline`	P2
`03-env`	`T-03-2`	實作 State/Action/Reward 轉換	MDP 數學公式	Python Env 邏輯	Random agent smoke test	`T-03-1`	P2
`04-agent`	`T-04-1`	建構 SB3 DQN 訓練腳本	`T-03-2` Env	`train.py` 腳本	腳本可啟動並執行 epoch	`03-env`	P3
`04-agent`	`T-04-2`	訓練 Agent 並執行評估	訓練腳本	Agent logs, 模型檔	檢視 Tensorboard Reward 曲線	`T-04-1`	P3
`05-report`	`T-05-1`	解析日誌並繪製成效比較圖表	Baseline/Agent logs	比較圖表 (`.png`)	檢查 throughput-RTT 權衡圖	`04-agent`	P4

---

12. Antigravity 開工指令草案

(可將以下內容直接複製至 Antigravity 的 prompt 中)

```
# Antigravity Implementation Instruction

You are Antigravity, acting as the Implementation Agent for this project. I am the Spec Ow
ner.
We are following a strict Spec-Driven Development workflow based on the OpenSpec methodolo
gy.

**CORE RULES:**

1. **Topic Frozen**: The project is "Deep Reinforcement Learning for Congestion Control an
d Throughput Optimization over a Single Bottleneck Link". You MUST NOT alter the topic.
2. **Scope Boundaries**: Do NOT implement IPFS, QUIC, traffic signal control, CityLearn, k
ernel-level TCP, or multi-agent routing. Stay exclusively within the ns-3 single bottlenec
k simulation.
3. **No Skipping**: You must execute the OpenSpec changes sequentially.
```

```
   - You MUST complete `01-project-charter` first.
   - You MUST complete `02-ns3-baseline-benchmark` before touching any Reinforcement Learn
ing code.
   - You MUST NOT jump directly to the DQN implementation.
4. **Execution Loop**: Every task must follow the **[ PLAN -> APPLY -> VERIFY ]** loop. Be
fore writing code, tell me your plan. After writing code, verify it works before declaring
it done.
5. **Spec Authority**: If you encounter issues, trigger the Fallback Plan defined in the s
pec. Any scope change must be returned to me (the Spec Owner) for confirmation.

**FIRST ACTION:**
Please acknowledge these instructions. Once acknowledged, read the `01-project-charter` sp
ecification and propose your `PLAN` for task `T-01-1`. Do not write code yet.
```

13. GitHub Repo 放置建議

在當前的 proposal submission 階段，建議建立以下 Repository 目錄結構以繳交給教授，展現專業的工程規畫能力。

```
drl-bottleneck-congestion-control/
├── README.md                  # 專案總覽 (放入 Executive Summary, 凍結題目與 Roadmap)
├── proposal/                  # 放置提案階段文件
│   ├── Final_Proposal.pdf     # 可先交的 PPT 匯出 PDF 版本
│   └── Final_Proposal.pptx    # 原始簡報檔
├── pdr/                       # 放置由 NotebookLM 產生的設計規格書
│   └── PDR_Core_Report.pdf    # PDR 規格化報告
├── assets/                    # 圖片與視覺資源
│   └── nblm_infographic.png   # 放置生成的 NotebookLM 資訊圖表
├── openspec-preview/          # OpenSpec 規格草案預覽 (proposal 階段核心亮點)
│   ├── 01-project-charter/
│   ├── 02-ns3-baseline-benchmark/
│   └── task_summary.md        # 放本文件的 task.md 總表
├── video/                     # 影片資料夾
│   └── 10_min_proposal.mp4    # 10 分鐘提案口頭報告影片
└── docs/                      # 額外參考文獻或筆記 (可選)
```

說明配置理由：

- **現在可以放什麼：** `proposal/` 簡報、`pdr/` 報告、`assets/` 資訊圖表、`video/` 提案影片，以及包含專案總覽與路線圖的 `README.md`。
- **哪些要等 implementation phase 才正式放：** 實際的程式碼目錄（如 `src/` 或 `simulation/`）、真正的 `openspec/` 實作工作區、訓練出的 DQN model weights 等。
- **為什麼使用 `openspec-preview/`：** 在 Proposal Submission 階段，我們尚未開始寫 Code。放一個 `openspec-preview/` (或將其視為靜態草案) 能夠向教授證明：**「我們沒有從想法直接跳到寫 code，而是已經把提案轉成了 AI 可執行的規格樹。」** 這能展現出極強的軟體工程素養與進度控管能力，且不會讓評審誤會我們已經在進行 Implementation。

14. 教授白板流程對應表

本表嚴格對應教授在白板上（IMG_5116.jpeg）所繪製的 DIC6 流程，確保專題推進符合指導教授之期望。

Whiteboard step	Meaning (含義)	NotebookLM output	GitHub artifact	Next tool	Status
`PPT → pdf`	確立簡報並轉為不可竄改格式	無 (由匯出功能處理)	`proposal/Final_Proposal.pdf`	NotebookLM	✅ 完成
`NBLM → PDR + 圖表`	透過 AI 將提案擴展為系統規格書	PDR Core Package, Infographic	`pdr/PDR_Core_Report.pdf`	OpenSpec	✅ 完成（PDR）
`PDR → OpenSpec`	將規格書拆解為 AI 可消化變更包	OpenSpec-ready Changes	`openspec-preview/`	Task breakdown	✅ 草案完成
`OpenSpec → task.md`	定義明確的工作任務與順序	`task.md` 草案總表	`task_summary.md`	Antigravity AI	✅ 草案完成
`task.md → code`	依據任務列表啟動程式碼撰寫	無 (AI 實作產出)	後續 implementation phase 產出 `src/`	程式碼執行器	⏳ Planned

Whiteboard step	Meaning (含義)	NotebookLM output	GitHub artifact	Next tool	Status
`Antigravity (Agent)`	執行 Spec Driven Development	開工指令草案	`README.md` (Workflow)	開發者 (審閱)	⏳ Planned
`Plan → Apply → Verify`	實作的閉環控制，避免偏航失控	Verification Checklist / Tasks	每一次 PR 或 Commit	下一個 Change	⏳ Planned

15. Final Verification Checklist

請在匯出本 Markdown 並部署至專案前，作最後檢核：

- [x] ~~**Proposal only**: 全文無「已完成訓練」、「結果顯示」等假造成效字眼。~~
- [x] ~~**PDR-ready**: 嚴格繼承了 本 PDR 中的 Scenario、Agent、Baselines、Metrics 規格。~~
- [x] ~~**No fake results**: 所有 Output、Chart 皆標示為「Expected」或「預期繪製」。~~
- [x] ~~**本 PDR used as source**: 內容緊扣 MDP 五元組與 TCP 基線等前置定義。~~
- [x] ~~**OpenSpec-ready changes complete**: 01 到 05 五個 Change 均有目的、Scope 與驗收標準。~~
- [x] ~~**task.md 草案 included**: 整理完成相依性列表，確認 `02-baseline` 早於 RL 開發。~~
- [x] ~~**Antigravity instruction included**: 提供了一段可直接貼給 AI 的「不准偏題」開工指令。~~
- [x] ~~**GitHub repo placement included**: 明確建議 `openspec-preview` 與提案文件的正確擺放位置。~~
- [x] ~~**PPT → PDR → OpenSpec → task.md → code workflow complete**: 流程表無縫對接教授白板。~~
- [x] ~~**Plan → Apply → Verify preserved**: 閉環驗證機制已寫入每個 Change 的 checklist 與 Antigravity 指令中。~~
- [x] ~~**No topic drift**: 已以 scope boundary 方式說明 IPFS、QUIC、多路徑與真實部署等方向不屬於本 proposal 階段。~~
- [x] ~~**Ready to export as Docs / PDF**: 格式相容，可直接貼入或匯出成文件。~~
- [x] ~~**Ready to place in GitHub**: 準備好作為期末提案的配套工程文件。~~